

# OPEN-ENDED LAB (OEL)

## OmniRetail Global Commerce Ecosystem

### Background Scenario

OmniRetail is a rapidly expanding multinational e-commerce organization operating across multiple countries, regional warehouses, payment gateways, shipping partners, and supplier networks. Over years of uncontrolled growth, the system evolved without proper governance — resulting in inconsistent business rules, overlapping application logic, incomplete normalization, and several operational anomalies across divisions.

The company now intends to redesign its data infrastructure. Each group has been hired as an independent database consulting firm. Because OmniRetail serves multiple regions and business divisions, every consulting group must produce a different implementation, uniquely driven by the Registration IDs of its members.

### Step 0 — Calculating Your Seed Number (Mandatory)

Every decision in this OEL, your database name, your extra entities, your data anomaly, depends on your group's unique Seed Number (SN). You must calculate it correctly before writing a single line of SQL.

#### How to Calculate SN

1. List the Registration IDs of ALL group members.
2. Extract every digit (0–9) from each Registration ID individually.
3. Sum all extracted digits together to get a Total.
4. Divide the Total by 7 and take the remainder. This is your Seed Number.

#### Worked Example

**Registration IDs: 50842, 50531, 50712**

Digit extraction:  $(5+0+8+4+2) + (5+0+5+3+1) + (5+0+7+1+2) = 19 + 14 + 15 = 48$

**Seed Number =  $48 \bmod 7 = 6$**

#### What the SN Controls

| SN | Mandatory Data Anomaly                                   | Extra Entity 1 | Extra Entity 2 |
|----|----------------------------------------------------------|----------------|----------------|
| 0  | Payment dates before order dates ( $\geq 15$ records)    | Coupons        | Reviews        |
| 1  | Negative transportation costs in Shipments               | Coupons        | Returns        |
| 2  | Customers with multiple loyalty cards                    | Discounts      | Reviews        |
| 3  | Negative stock values in Warehouse Inventory             | Discounts      | Claims         |
| 4  | Order totals $\neq$ payment amounts ( $\geq 15$ records) | Subscriptions  | Reviews        |
| 5  | Employees appearing as their own manager                 | Subscriptions  | Claims         |
| 6+ | Suppliers linked to non-existent Regions                 | Returns        | LoyaltyPoints  |

### Step 1 — Database Naming Convention

The database name must be:

**Retail\_<SeedNumber>**

Example: if SN = 4 → database is named Retail\_4

**Mandatory Comment Block:**

Every SQL script must begin with a comment block that includes: all Registration IDs, the digit extraction for each ID, the full summation, and the final Seed Number. Scripts without this block will not be marked.

## Step 2 — Schema Design

### 2.1 Core Entities (Mandatory for ALL Groups)

Your schema must contain at minimum the following nine entities. You must define all primary keys, foreign keys, and appropriate constraints.

| Entity     | Minimum Attributes Expected                                                           |
|------------|---------------------------------------------------------------------------------------|
| Customers  | CustomerID (PK), FullName, Email, Phone, DateOfBirth, RegistrationDate, RegionID (FK) |
| Orders     | OrderID (PK), CustomerID (FK), OrderDate, TotalAmount, Status, WarehouseID (FK)       |
| Products   | ProductID (PK), ProductName, Category, UnitPrice, SupplierID (FK), StockQuantity      |
| Warehouses | WarehouseID (PK), WarehouseName, RegionID (FK), Capacity, ManagerEmployeeID (FK)      |
| Suppliers  | SupplierID (PK), SupplierName, ContactEmail, RegionID (FK), ContractStartDate         |
| Payments   | PaymentID (PK), OrderID (FK), PaymentDate, Amount, PaymentMethod, Status              |
| Shipments  | ShipmentID (PK), OrderID (FK), ShipDate, DeliveryDate, TransportCost, CarrierName     |
| Employees  | EmployeeID (PK), FullName, JobTitle, ManagerID (FK self-ref), HireDate, RegionID (FK) |
| Regions    | RegionID (PK), RegionName, Country, Currency                                          |

### 2.2 Additional Entities Based on Your SN

The Seed Number further determines two additional entities that become mandatory components of the implementation.

Groups having Seed Number zero must implement Coupons and Reviews.

Groups having Seed Number one must implement Coupons and Returns.

Groups having Seed Number two must implement Discounts and Reviews.

Groups having Seed Number three must implement Discounts and Claims.

Groups having Seed Number four must implement Subscriptions and Reviews.

Groups having Seed Number five must implement Subscriptions and Claims.

Groups having Seed Number six and onwards must implement Returns and LoyaltyPoints.

Groups are free to add additional entities if required by their business assumptions.

The database must be populated manually using SQL INSERT statements. At least twenty-five customers, forty orders, thirty products, forty payment transactions, twenty employees and ten warehouses must be inserted.

To ensure dataset uniqueness, every group member's Registration ID must appear indirectly inside the inserted data. Registration numbers may be embedded within product identifiers, warehouse identifiers, customer codes, invoice numbers, shipment tracking numbers or any other meaningful attributes. The same embedding strategy may not be repeated throughout the dataset.

Real-world information systems rarely contain perfectly clean data. Therefore, groups must intentionally introduce inconsistencies into their datasets according to their Seed Number.

Groups with Seed Number zero must create at least fifteen records where payment dates occur before order dates.

Groups with Seed Number one must introduce shipment records containing negative transportation costs.

Groups with Seed Number two must create customers associated with multiple loyalty cards.

Groups with Seed Number three must generate warehouse inventories having negative stock values.

Groups with Seed Number four must ensure that at least fifteen orders have payment amounts different from order totals.

Groups with Seed Number five must create employees who appear as their own managers.

Groups with Seed Number six must assign suppliers to regions that do not exist within the Regions table.

These inconsistencies should not be corrected during insertion because they represent operational problems that later analytical queries are expected to identify and investigate.

## 2.3 Integrity Constraints

- Every table must have a defined PRIMARY KEY.
- All cross-table references must use FOREIGN KEY constraints.
- At least two tables must include CHECK constraints enforcing a business rule.
- At least three columns across the schema must use meaningful DEFAULT values.
- At least one UNIQUE constraint must appear on a non-PK column.

# Step 3 — Personalized Data Population

## 3.1 Minimum Record Counts

| Entity     | Minimum Records |
|------------|-----------------|
| Customers  | 25              |
| Orders     | 40              |
| Products   | 30              |
| Payments   | 40              |
| Employees  | 20              |
| Warehouses | 10              |
| Shipments  | 30              |
| Suppliers  | 10              |
| Regions    | 5               |

## 3.2 Registration ID Embedding

Every group member's Registration ID must be traceable inside the dataset. You must embed each Registration ID into a different column/entity using the rules below:

- Registration IDs may be embedded as: a product code suffix, a warehouse code prefix, a customer reference code, an invoice number, a shipment tracking number, an employee badge number, or a supplier contract ID.
- The same embedding strategy may NOT be reused across two different Registration IDs.
- Embeddings must be logical, a product code like PRD-50842 or an invoice INV-50531 are acceptable; putting a Reg ID as someone's phone number is not.
- Your comment block must include a mapping table: RegID → Embedded In → Column → Sample Value.

### 3.3 Mandatory Data Anomalies (SN-Specific)

Refer to the Seed Number table. You **MUST** introduce the anomaly assigned to your SN into the dataset. These anomalies are intentional, they represent operational problems that your Task 1 query will later detect.

- Anomalies must be embedded within INSERT statements, not added later.
- Anomalies must be realistic, if your SN requires payment dates before order dates, the difference should vary (some 1 day, some 3 months) to simulate real-world disorder.
- Do NOT add a comment pointing to each anomalous record. The query in Task 1 must find them analytically.
- Normal (non-anomalous) records must coexist in the same tables so the anomaly is not trivially detectable.

## Task 1 — Advanced Anomaly Detection

Management suspects that certain products are associated with unusual purchasing patterns but does not know which products are responsible. You are required to write a single SQL query that identifies products satisfying **ALL** of the following conditions simultaneously:

| #  | Condition                  | Description                                                                                                                                                                                        |
|----|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| C1 | Never purchased            | The product does not appear in any completed, confirmed order from any customer.                                                                                                                   |
| C2 | Linked to anomaly          | The product appears exclusively within the anomalous transactions introduced according to your group's Seed Number. (Define what 'anomalous' means for your SN and express it as a SQL condition.) |
| C3 | Top 20% price bracket      | The product's UnitPrice falls within the most expensive 20% of all products in the system.                                                                                                         |
| C4 | Above-average customer age | Customers who have purchased this product's category have an average age greater than the company-wide average customer age.                                                                       |

### Technical Constraints for Task 1

- The query must execute correctly on your own dataset and return a non-empty result.
- If your dataset naturally has no qualifying product, you must adjust your data to ensure at least one product qualifies, and document this adjustment.

## Task 2 — Tri-Approach Analytical Query

Database engineers solve identical problems using completely different techniques. You must choose **ONE** analytical business problem from the list below and solve it three times, each solution using a completely different SQL mechanism:

| Option | Business Problem                                                                                                                  |
|--------|-----------------------------------------------------------------------------------------------------------------------------------|
| A      | Find all customers who have spent more than the average order value across their own region (not the global average).             |
| B      | Identify products that have been ordered in every region where the company operates.                                              |
| C      | For each warehouse, find the employee with the highest total value of orders they were responsible for fulfilling.                |
| D      | Find the month in which payment collection was most inconsistent — measured as the highest standard deviation of payment amounts. |
| E      | Identify suppliers whose products have never been involved in a shipment with a delivery delay of more than 7 days.               |

## The Three Required Approaches

| Approach  | Allowed SQL Mechanism        | What is Prohibited                                                        |
|-----------|------------------------------|---------------------------------------------------------------------------|
| Version 1 | JOIN operations only         | No subqueries of any kind, no window functions, no CTEs                   |
| Version 2 | Correlated subqueries only   | No JOINS between the main query and external tables, no window functions  |
| Version 3 | Window functions exclusively | No JOINS in the aggregation logic, no correlated subqueries in the filter |

## Additional Requirements for Task 2

- State clearly which option (A–E) you have chosen.
- Each version must return identical results (same rows and columns).
- Include a brief written comparison (3–5 lines) discussing the performance implications of each approach and which you would recommend for a large production dataset.

Students must implement a stored procedure named `CreateOrder()`.

- The procedure accepts a customer identifier, a product identifier and a quantity as parameters.
- The procedure must verify stock availability, reduce inventory, generate a payment record, create a shipment entry and assign an employee responsible for order fulfillment.

An audit mechanism is required to monitor financial modifications.

- Students must create a trigger executing after every update on the Payments table.
- The trigger should record the old amount, new amount, order identifier, current database user and execution timestamp inside an AuditLog table.
- However, when payment updates originate from the `CreateOrder()` procedure, duplicate audit entries must not be generated.
- No instructions are provided regarding how this problem should be solved.

## Submission Requirements

- A single .sql script containing: schema DDL, all INSERT statements, stored procedure, trigger, AuditLog table, and all analytical queries.
- A Document report containing: the SN calculation, the Registration ID embedding map, your 3NF justification, Task 2 performance comparison, and screenshots of all query outputs.
- The PDF report must include screenshots of `CreateOrder()` test outputs and AuditLog table contents after the three test cases.

### **⚠ ACADEMIC INTEGRITY NOTICE**

No AI system can produce a correct submission without knowing your specific IDs, and even then, the IDs must be correctly embedded in the data in a traceable way that will be verified during the viva. Any submission that does not match your group's Seed Number will receive zero marks and will be referred for misconduct review.